PS-033: DNS-Based Discovery of Diagnostic Services

Discovery Client (Student B)

Purpose

The discovery client finds the three diagnostic services (latency, traceroute, loss/jitter) through signed DNS records instead of hardcoded addresses. It then checks that the metadata is genuine and calls each service with an authenticated request. It keeps two questions apart: whether a record is authentic (DNS) and whether the caller is allowed to use the service (the service itself).

Processing flow

For each service name, the client performs four steps and stops at the first failure.

1. **Resolve.** It queries the SVCB record for `<service>.lab.test` through the validating Unbound resolver (127.0.0.1:5301), with the DNSSEC OK bit set.
2. **Validate.** It accepts the answer only if the resolver sets the AD (Authenticated Data) flag. A DNSSEC failure comes back from the resolver as SERVFAIL, and the client refuses to continue.
3. **Check the metadata.** It reads the port and address hint from the SVCB record, and a SHA-256 digest of the service's descriptor from a private-use SVCB key (`key65400`). It then fetches the descriptor from `/.well-known/descriptor.json`, hashes the exact bytes received, and compares the two values.
4. **Invoke.** Only after the digest matches does it call the service's `/probe` endpoint with a bearer token. The service decides whether to accept the call.

Because the digest check comes before the call, a service whose descriptor has been altered never receives the client's token.

Outcome classification

Every failure is reported as a typed outcome, so errors are visible and never silently ignored.

Outcome	Trigger
VALID	DNSSEC-validated record, matching digest, authorised call succeeded
BOGUS	The resolver refused the name (SERVFAIL, DNSSEC validation failed)
INSECURE	Answer received without the AD flag
MALFORMED	Missing or unusable SVCB data (no port, no digest, digest not 64 hex characters)
DIGEST_MISMATCH	Descriptor hash differs from the digest published in DNS
UNAVAILABLE	Resolver or service unreachable, or an unexpected HTTP error
UNAUTHORISED	The service rejected the token (HTTP 401 or 403)

An unreachable resolver is reported as UNAVAILABLE and never as BOGUS, so an outage cannot be mistaken for an attack.

Separation of integrity and authorisation

DNSSEC and the descriptor digest show that the published metadata came from the zone owner and was not altered. They do not show that the service is safe or that the caller may use it. The token check is enforced by each service on every request, so finding an endpoint never grants access to it.

Testing

A scripted test file (`src/client/test_cases.py`) exercises every outcome and prints a PASS/FAIL table. Some cases are simulated inside the client (a SERVFAIL answer, malformed records, an unreachable resolver). The live cases run against the real stack: the three services, and two deliberately faulty zone entries published by the DNS side, `bogus.lab.test` (broken chain of trust) and `malformed.lab.test` (signed SVCB without a port).

Run it with the stack up:

```
cd src/client
py test_cases.py --live
```

Case	Expected	Result
Valid call with token	VALID	PASS
Wrong token	UNAUTHORISED	PASS
Service down (closed port)	UNAVAILABLE	PASS
Malformed SVCB records	MALFORMED	PASS
Descriptor digest mismatch	DIGEST_MISMATCH	PASS
Resolver answers SERVFAIL	BOGUS	PASS
Resolver unreachable	UNAVAILABLE	PASS
Live DNS discovery: latency	VALID	PASS
Live DNS discovery: traceroute	VALID	PASS
Live DNS discovery: lossjitter	VALID	PASS
Live faulty zone: bogus	BOGUS	PASS
Live faulty zone: malformed	MALFORMED	PASS

Result: 12 of 12 cases passed.

(Add the screenshot of the final test table here.)

Scope against the proposal

The proposal (Optimization slide) lists several optimisations. Status in the submitted prototype:

- **Fail closed on any validation error:** implemented. Every failure produces a typed outcome and the client never sends its token after a failed check.
- **Dynamic re-resolution when endpoints move:** implemented. The client resolves each service on every run, so updating one SVCB record is enough.
- **Parallel SVCB + TLSA queries and priority fallback:** not implemented; see Future work.
- **TTL-based caching of validated results:** not implemented; see Future work.

Future work

The following items from the proposal were deferred to keep the prototype focused on discovery, validation and the failure cases:

- **TLSA/DANE and TLS:** publish TLSA records and serve the mock services over TLS, so the client can pin the service's key through DNS.
- **Parallel SVCB and TLSA queries:** query both record types concurrently to reduce discovery latency.
- **Priority fallback:** try further SVCB records in priority order when the first endpoint is unavailable. The prototype uses the first record only.
- **TTL caching:** cache validated discovery results for the record's TTL, with care that a cache does not hide later failures.

Other limitations

- The descriptor digest is carried in a private-use SVCB key chosen by the team. A production system would use a registered parameter.
- The prototype calls the services over plain HTTP. Authenticity of the metadata comes from DNSSEC and the digest, not from TLS.